

Claude Code Agent Teams

Operating guide — installing the 21-agent team into a project, running it, and what you must supply for top-level output.

company-claude-os-v2 · August 2026 · Batore

The one thing to understand before anything else. Every teammate starts with an **empty context window** and none of your conversation history. It has not read your chats. It has not seen your other projects. What it knows is: `CLAUDE.md`, the rules files, the plan, and its spawn prompt.

Everything in this guide follows from that. Output quality is capped by what you write down.

Part 1 · Add the team to a project

STEP 1 — KEEP THE PACK SOMEWHERE PERMANENT

Extract the zip once. Move it out of Downloads so a cleanup can't delete it.

```
Move-Item "$env:USERPROFILE\Downloads\company-claude-os-by-claude" "$env:USERPROFILE\.claude-os"
```

STEP 2 — MAKE SURE THE PROJECT IS A GIT REPO FIRST

If it's a brand-new project, scaffold it and make one commit *before* any agent touches it. Without a clean baseline the hooks can't tell what changed, and you have no rollback point.

```
git init
git add -A
git commit -m "baseline before agent team"
```

STEP 3 — RUN THE INSTALLER (ONE LINE, EVERY PROJECT)

Open the project in VS Code, launch Claude Code, and paste:

```
Read INSTALL.md at C:\Users\sriva\.claude-os and execute it fully.
Don't ask me to confirm steps.
```

It detects your stack from real dependency files, copies 21 agents / 8 rules / 3 hooks, merges settings rather than overwriting, adds stack-appropriate permissions, verifies every ownership path exists, and installs gstack. It is idempotent — safe to re-run.

STEP 4 — RUN BOOTSTRAP.SH (CLAUDE CODE OFTEN CANNOT DO THIS ON WINDOWS)

From your project root, in WSL2 or Git Bash — not PowerShell:

```
bash bootstrap.sh
```

It makes the three hooks executable and verifies the cost guard actually fires. It must print **ENFORCEMENT LAYER LIVE**. If it doesn't, it tells you exactly what failed.

If this step fails, you have no enforcement layer. All three quality gates — including the one blocking paid API re-runs — silently never fire. The setup will look installed and won't be. Don't skip the verification line.

STEP 5 — VERIFY, THEN RESTART

```
(Get-ChildItem .\.claude\agents).Count      # 21
(Get-ChildItem .\.claude\rules).Count       # 8
(Get-ChildItem .\.claude\hooks).Count      # 3
Get-Content .\.claude\settings.json | ConvertFrom-Json | Out-Null; echo "json ok"
```

Then close Claude Code completely and reopen it. Settings load only at startup.

STEP 6 — CONFIGURE AND SMOKE TEST

Run `/config` and set **Default teammate model to Sonnet**. Lead stays Opus — teammates are where cost multiplies.

```
Spawn a teammate named scout to list the top-level files in this repo.
```

A `scout` row should appear in the panel below your prompt box. If nothing appears, the feature flag didn't load — recheck `\.claude\settings.json`.

STEP 7 — SEMANTIC CODE SEARCH

```
/sync-gbrain
```

Every teammate starts blank and greps around to orient. Four doing that simultaneously is exactly the workload this pays for. On client repos, set the trust tier to read-only so one client's patterns never write into another's.

Part 2 · What you write down, once per project

2.1 — CLAUDE.md: the domain section

The installer fills in stack and commands correctly, because it reads real config files. It **cannot** know what your system means. That part is yours, and it's the highest-value ten minutes in the whole system.

```
## Domain
~2,000 registered students across multiple branches.
Memberships, food passes, attendance, WhatsApp integration,
cashback, referral analytics. ~90 backend actions, 25+ tables.

A "food pass" is a purchasable entitlement, not a boolean flag.
Attendance drives cashback eligibility.

## Constraints
Mid-migration off Supabase Cloud to self-hosted VPS.
Check before writing connection or client code.
Do not refactor the legacy attendance module – known debt, deliberate.
```

The part people skip: what entities really mean. Without "a food pass is a purchasable entitlement, not a boolean flag," an agent models it as a flag — and every downstream decision inherits that mistake. This is the difference between code that is wrong and code that is confidently wrong.

The test

Hand `CLAUDE.md` to a competent engineer who has never seen the project. Could they set it up, run the tests, and know which directory not to touch? If not, teammates can't either.

2.2 — DESIGN.md (only if the project has a UI)

```
/design-consultation
```

It asks about typography, palette, spacing, and motion. **Answer with actual decisions, not preferences.** "Modern and clean" is not a constraint the agents can act on. "Base 16px, 1.25 scale, these six greys, 4px spacing unit, 200ms ease-out" is.

Without this, output drifts to the statistical centre of everything the model has seen — gradient hero, three-column icon grid, uniform border radius, centred body text. Not laziness; that's simply what unconstrained generation produces.

Once per product, then permanent.

2.3 — Verify the ownership map

The installer writes one. Check that every path actually exists:

```
Test-Path .\src\components, .\src\pages, .\supabase\functions, .\supabase\migrations, .\tests
```

Any `False` means that ownership line is wrong. A glob pointing at a non-existent directory silently protects nothing.

Part 3 · Starting the team

The two rules that prevent lost work

Rule	Why
One browser owner per mission	The gstack browse daemon is a single Chromium session. Cookies and tabs carry over. Two agents driving it corrupt each other and generate bugs that don't exist.
Auto-committing skills only in the lead, after teammates shut down	<code>/review</code> , <code>/qa</code> , <code>/design-review</code> , <code>/ship</code> , <code>/land-and-deploy</code> fix and commit autonomously. Run them while teammates hold files and you get a history nobody can bisect.

Your first run — Mission 1, always

Open `docs\MISSIONS.md`, copy Mission 1, fill in a real feature name, paste it. Four teammates review the plan simultaneously from four angles and argue with each other.

All four are read-only by tool allowlist — they physically have no Write or Edit tool. That restriction is enforced by the platform, not by asking them nicely. Nothing can be damaged while you learn the controls.

Do not start with Mission 4. That's the only mission where four agents write to the same repo concurrently. You want the ownership-map habit before then.

Controls while a team runs

Key	Action
↑ / ↓	Select a teammate in the panel below your prompt
Enter	Open that teammate's transcript; type to send it instructions
Esc	Back to the lead session
Ctrl + T	Toggle the shared task list
x	Stop the selected teammate

Common stall: the lead decides everyone is done and starts implementing itself. If you see that, type: *"Wait for your teammates to complete their tasks before proceeding."*

Part 4 · What you supply every single feature

Input 1 — The problem, not your solution

Given to `/office-hours`, run solo.

Weak: "Add a referral cashback screen with a table and filters."

Strong: "Students refer friends but we can't tell which referrals convert to paid memberships, so we can't decide whether to keep funding the cashback scheme."

The second lets the strategist find a better answer than a screen. The first pre-commits you to a screen.

Input 2 — The commercial frame

One line, stated explicitly, every time. The system defaults wrong without it.

Situation	Say
Client work, fixed price	<code>HOLD SCOPE</code> — otherwise the strategist expands and eats your margin
Your own product	<code>SCOPE EXPANSION</code> — otherwise it holds back where ambition is the point
Real deadline	Say so, with the date

Input 3 — Two checks on the ownership map

Mission 1 produces the map. Before Mission 4, check exactly two things:

- Does every listed folder actually exist?
- Is any folder listed twice?

These are the only two failure modes that *destroy work* rather than merely producing bad output.

Input 4 — Sequencing in the spawn prompt

Teammates share no history, so you must state who unblocks whom. Mission 4 already contains these lines — keep them when you edit:

```
schema publishes the final table shape to api before api starts.
api publishes the endpoint contract to ui before ui builds against it.
qa tests against the acceptance criteria, not the implementation.
```

Part 5 · The loop, every feature

#	Do this	Notes
1	<code>/office-hours</code>	Solo. A team adds nothing to a conversation. Bring Inputs 1 and 2.
2	Mission 1 — plan review	4 teammates, read-only. CEO, design, DevEx, architecture lenses arguing.
3	Check the ownership map	Input 3. Two checks. Do not skip.
4	Mission 4 — build	4 teammates. Each runs <code>/freeze</code> first.
5	Wait for full shutdown	Confirm every teammate has stopped.
6	Mission 2 — review	4 lenses: staff, security, cross-model, performance. Read-only.
7	<code>/review</code> <code>/qa</code>	Lead only, now safe to auto-fix — nobody else holds files.
8	<code>/ship</code> then <code>/land-and-deploy</code>	Always solo. Never as a team.
9	Mission 6.5 — codify	What did an agent reason through that should have been a script? Write it.

Step 9 is what makes the system improve rather than merely run. Reasoning is probabilistic; scripts are not. Five chained steps at 90% accuracy leaves you at 59%. Every mechanical sequence you move into `tools/` removes a probabilistic step permanently. If `tools/` is still empty in a month, the system is running but not getting better.

Part 6 · Choosing the right mission

Situation	Use
New product idea	Solo <code>/office-hours</code>
Routine plan review	Solo <code>/autoplan</code>
High-stakes plan review	Mission 1 — 4 teammates
Routine PR review	Solo <code>/review</code>
Pre-release or security-sensitive	Mission 2 — 4 teammates
Bug with an obvious cause	Solo <code>/investigate</code>
Bug nobody can explain	Mission 3 — 5 rival hypotheses
Single-layer feature	Solo, sequential
Cross-layer feature, independent files	Mission 4 — 4 teammates
Infrastructure migration	Mission 6 — 3 teammates under <code>/guard</code>
Anything that pushes, merges, deploys	Always solo
A mechanical step repeated twice	Write a tool — not another agent
Ten unrelated features at once	Neither — parallel worktrees, not agent teams

Part 7 · When output degrades

Symptom	Cause	Fix
Code looks right, is wrong	Thin CLAUDE.md domain section	Add business meaning, not just schema
UI looks AI-generated	No DESIGN.md	<code>/design-consultation</code> , commit real values
Teammates overwrote each other	Ownership globs missing, overlapping, or pointing nowhere	Verify every path exists; one glob per role
Scope grew, margin gone	Strategist not told it's fixed-price	Say <code>HOLD SCOPE</code> explicitly
Frontend built against wrong API	No sequencing in the spawn prompt	Add the "publishes before" lines
Reviewer found nothing	No acceptance criteria to review against	Criteria belong in the plan, observable and falsifiable
Quality gates never fire	Hooks not executable	Run <code>bash bootstrap.sh</code> from WSL2 or Git Bash
Phantom bugs in QA	Two agents on the browser daemon	One browser owner per mission, always
Commit history unbisectable	Auto-committing skill ran in a teammate	Lead only, after shutdown

Part 8 · Limits worth knowing

- **Teammates cannot be resumed.** `/resume` and `/rewind` don't restore them. Run `/context-save` as the closing ritual of any mission that took real effort.
- **One team per session, no nested teams.** Teammates can't spawn their own teammates. The lead is fixed for the session.
- **Task status can lag.** Teammates sometimes fail to mark tasks complete, blocking dependents. Check and nudge.
- **Teams can form unasked.** Once enabled, a subagent Claude names on its own launches as a teammate.
- **Cost scales linearly per teammate.** Each has its own context window. Start at three.
- **React Native projects:** the browse daemon can't reach the app. Agent 12 and every browser skill are inapplicable — use Maestro or Detox instead.
- **Off switch:** set `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` to `"0"` in `.claude/settings.json`. No reinstall needed.

Part 9 · What the team can and cannot do

Does well	Does not do
Parallel review across independent lenses — the overlap between reviewers is high-confidence signal	Decide what is worth building
Competing-hypothesis debugging — breaks the anchoring that makes a single agent settle on the first plausible cause	Originate a visual identity. It holds a line very well; it does not draw the line
Cross-layer implementation where files genuinely don't overlap	Know your client's tolerance, market, or what they'll hate
Consistent enforcement of standards across every file, without fatigue	Reliably know what it doesn't know — it produces fluent wrong answers in the same tone as right ones
Documentation and handover packs generated from actual code	Want your project to succeed. It completes the task as described — describe it wrong and it completes it wrongly

The compression: you supply domain knowledge, taste, and sequencing. The team supplies consistency, breadth, and enforcement. It genuinely cannot supply the first three — and every failure mode in Part 7 is one of them missing.

Quick reference

When	Do
New project	git commit baseline → run INSTALL.md one-liner → <code>bash bootstrap.sh</code> → fill CLAUDE.md domain → <code>/design-consultation</code> → <code>/sync-gbrain</code>
New feature	<code>/office-hours</code> → Mission 1 → check ownership map → Mission 4 → shutdown → Mission 2 → <code>/review</code> <code>/qa</code> <code>/ship</code> → Mission 6.5
Every time, state	The problem (not the solution) · the commercial frame · the sequencing
Every time, check	Ownership paths exist · no folder listed twice
Never	Ship with a team · two agents on the browser · auto-commit skills in a teammate

Mechanics verified against code.claude.com/docs/en/agent-teams and the gstack skill list as of August 2026. Agent teams is experimental and gstack ships changes frequently — re-verify before relying on any specific behaviour. Nothing in this pack has been run on a production repo yet; the first real mission will surface things this guide does not predict.